Reporte Técnico — MiniShell: Intérprete de Comandos en ARM-64 Puro

Arquitectura de Computadores y Ensambladores 1 · Proyecto 2 · Segundo Semestre 2026
Universidad de San Carlos de Guatemala, Facultad de Ingeniería, Ingeniería en Ciencias y Sistemas

Autor: Angel Daniel Ruiz Ramos · **Carné:** 202505232 · **Fecha de entrega:** 23/10/2026

1. Introducción

MiniShell es un intérprete de comandos escrito íntegramente en ensamblador ARM-64 (GNU Assembler, GAS) para Linux. No usa lenguaje C ni bibliotecas externas: el ejecutable se enlaza con `ld` sin libc y toda la interacción con el sistema operativo se hace mediante syscalls del kernel Linux.

El shell muestra el prompt `OMEGA_TERMINAL>`, lee una línea, la separa en comando y argumentos, y ejecuta una de dos rutas: un comando interno (`help` , `clear` , `exit` , `echo`) o un programa externo del sistema (`ls` , `pwd` , `date` , etc.) mediante `clone` (fork) y `execve` .

Este reporte explica las decisiones de diseño, el uso del stack, las syscalls empleadas, el manejo de banderas y registros, las pruebas realizadas y la evidencia de depuración con GDB.

2. Entorno y compilación

Elemento	Detalle
Plataforma	Linux ARM64 (nativo) o x86-64 con QEMU user-mode (`qemu-aarch64`)
Ensamblador	`aarch64-linux-gnu-as` (GAS), con `-g` para símbolos de depuración
Enlazador	`aarch64-linux-gnu-ld` , sin libc, punto de entrada `_start`

Elemento	Detalle
Depurador	`gdb-multiarch` conectado a QEMU por el puerto 1234
Automatización	`Makefile` (un solo comando: `make`)

`practica.s` es el archivo principal y usa `.include` para incorporar los demás módulos, de modo que se genera un único `practica.o` y de ahí el ejecutable `programa`:

```
make            # as -g practica.s -o practica.o ; ld practica.o -o programa
make run        # ejecuta con qemu-aarch64
make debug      # QEMU esperando a GDB en :1234
make gdb        # abre GDB con el script s.gdb
make test       # corre pruebas.sh (20 pruebas automáticas)
make clean
```

En una máquina ARM64 nativa se usa `make CROSS= QEMU=`. El Makefile declara como dependencias de `practica.o` todos los `.s` incluidos, por lo que un cambio en cualquier módulo provoca la recompilación.

3. Estructura del proyecto

Archivo	Contenido
`practica.s`	`_start` (ciclo principal), `read`, `verificar_largo`, `print`, `exit`, `error`, mensajes y buffers
`leer_comando.s`	`intrucion_init`: parsing de la línea en comando y mensaje
`interpretar.s`	`comparar`: despacho a builtins o a comando externo
`help.s`, `echo.s`, `clear.s`	Comandos internos
`fork.s`	`fork`: creación del proceso hijo con `clone`
`execve.s`	`execve`, `preparar_argumentos`, `construir_ruta`, `llamar_execve`
`wait.s`	`wait`: espera al hijo con `wait4`

Archivo	Contenido
`Makefile`, `README.md`, `pruebas.sh`, `s.gdb`	Compilación, documentación, pruebas y sesión de depuración

La división en módulos sigue el flujo lógico del shell (entrada, parsing, despacho, ejecución) y permitió probar cada función por separado antes de integrarla.

4. Diseño general

4.1 Flujo del ciclo principal

`_start` implementa el ciclo principal. En pseudocódigo:

```
ciclo:
    print(prompt)
    x16 = comando_buffer ; x17 = mensaje_buffer
    n = read()                          -> n == 0 (EOF): imprimir \n y exit
    n = verificar_largo(n)              -> n < 0 (línea demasiado larga): volver
a ciclo
    intrucion_init(n)                   -> x18 = largo del comando, x19 = largo
del mensaje
    si x18 == 0: volver a ciclo         (línea vacía)
    comparar()                          -> builtin o programa externo
    volver a ciclo
```

El ciclo es una etiqueta a la que se regresa con `b _start`. Como `_start` es el nivel más alto y ninguna función deja datos en el stack al retornar, el stack siempre está balanceado al volver al prompt.

4.2 Memoria

Los datos de texto viven en `.data` y los buffers de trabajo en `.bss`:

Símbolo	Tamaño	Uso
`input_buffer`	64 B	Línea cruda leída de stdin (máx. 63 bytes por lectura)
`comando_buffer`	64 B	Primera palabra de la línea (el comando)

Símbolo	Tamaño	Uso
`mensaje_buffer`	64 B	Resto de la línea (los argumentos)
`drain_byte`	8 B	Un byte para descartar el resto de una línea demasiado larga
`argumentos`	520 B	Arreglo `argv` (punteros de 8 bytes) para `execve`
`ruta_buffer`	80 B	Ruta completa `/bin/<comando>`

4.3 Convención de registros propia del programa

Para mantener el código legible se fijó un mapa de registros que se documenta en los comentarios de `practica.s`:

Registro	Uso
`x0`, `x1`, `x2`, `x8`	Argumentos y número de syscall (convención de Linux ARM64)
`x16`	Dirección de `comando_buffer` (fija durante el ciclo)
`x17`	Dirección de `mensaje_buffer` (fija durante el ciclo)
`x18`	Longitud del comando (0 indica línea vacía)
`x19`	Longitud del mensaje
`x22`	Puntero a `input_buffer` durante el parsing
`x9` – `x15`	Temporales (índices, bytes leídos, punteros a cadenas constantes)

5. Descripción de las funciones

5.1 `read` (practica.s)

Lee con la syscall `read(0, input_buffer, 63)`. Si el resultado es negativo salta a `error`; en caso contrario escribe un byte NUL en `input_buffer[x0]` para terminar el texto y devuelve en `x0` la cantidad de bytes leídos. Un valor de 0 significa EOF (Ctrl+D). Se leen como máximo 63 bytes para dejar espacio al terminador.

5.2 `verificar_largo` (practica.s)

Detecta una línea más larga que el buffer. Si `read` devolvió menos de 63 bytes la línea cabe. Si devolvió exactamente 63, revisa si el byte en la posición 62 es el salto de línea (62 caracteres más `\n` es una línea válida). En caso contrario, descarta el resto de la línea leyendo de a un byte hasta encontrar `\n`, imprime el aviso `Error: la linea excede 62 caracteres` y devuelve `-1` en `x0`. Sin este descarte, el resto de la línea quedaría en el buffer del terminal y se interpretaría como un comando nuevo.

5.3 `intrucion_init` (leer_comando.s)

Es el parser. Recorre `input_buffer` en tres fases:

1. Salta espacios (32) y tabuladores (9) iniciales.
2. Copia el primer token a `comando_buffer` hasta encontrar espacio, tab, `\n` o NUL, con un tope de 63 caracteres. El conteo queda en `x18`.
3. Salta los separadores siguientes y copia el resto de la línea a `mensaje_buffer`, con el conteo en `x19`.

Al final ambos buffers quedan terminados en NUL. Si la línea solo tiene espacios o Enter, `x18` queda en 0 y `_start` vuelve a mostrar el prompt sin ejecutar nada.

5.4 `comparar` (interpretar.s)

Es el despachador. Para cada builtin (`echo`, `clear`, `help`, `exit`) primero compara la longitud del comando con la de la cadena constante (una comparación rápida con `x18`) y solo si coincide compara byte a byte. Si algún builtin coincide llama a su función; si ninguno coincide salta a la etiqueta `Externo`, que llama a `fork`.

5.5 Comandos internos

- **`help`** imprime la lista de comandos internos con `print`.
- **`echo`** imprime `mensaje_buffer` (`x17` , longitud `x19`) y luego un salto de línea. Los espacios y tabs múltiples entre `echo` y el texto ya fueron eliminados por el parser.
- **`clear_Command`** imprime la secuencia ANSI `ESC[2J ESC[H` (borra la pantalla y mueve el cursor a la esquina superior izquierda). No requiere una syscall especial: es una escritura normal a stdout interpretada por el terminal.
- **`exit`** llama a la syscall `exit(0)`.

5.6 `fork` (fork.s)

ARM64 no tiene la syscall `fork` (existe solo `clone`), por lo que se usa `clone(SIGCHLD, 0, 0, 0, 0)` con el número 220. Con `flags = SIGCHLD (17)` el kernel crea un proceso que se comporta como un `fork` clásico. El resultado en `x0` se evalúa con las banderas de un solo `cmp x0, #0`:

```
cmp  x0, #0
b.lt error_fork        // N != V: valor negativo, falló la creación
b.eq proceso_hijo     // Z = 1: x0 == 0, somos el hijo
                      // en otro caso x0 > 0: padre, x0 = PID del hijo
```

- **Hijo:** llama a `execve`. Si `execve` regresa es que falló: imprime `No se pudo ejecutar el comando` y termina con `exit(1)`.
- **Padre:** llama a `wait` y retorna al despachador.
- **Error de `clone`:** imprime `No se pudo crear el proceso hijo` y retorna.

5.7 `execve` y funciones auxiliares (execve.s)

- **`preparar_argumentos`** construye `argv` en el arreglo `argumentos`: `argv[0]` apunta al comando (`x16`); cada palabra de `mensaje_buffer` pasa a ser `argv[1..n]` (se escribe un NUL en el separador, sobre el mismo buffer); el arreglo termina con un puntero NULL, como exige `execve`.
- **`execve`** busca el carácter `/` en el comando. Si lo contiene (por ejemplo `/bin/echo`) lo usa como ruta tal cual; si no, llama a **`construir_ruta`**, que copia el prefijo `/bin/` y el nombre del comando en `ruta_buffer`.
- **`llamar_execve`** ejecuta la syscall `execve(ruta, argv, NULL)` con el número 221.

5.8 `wait` (wait.s)

Ejecuta `wait4(pid, NULL, 0, NULL)` (syscall 260). Guarda el PID en `x9` porque la syscall devuelve su resultado en `x0`. Si el retorno es `-4` (EINTR, espera interrumpida por una señal) repite la llamada; esa comparación se hace con `cmn x0, #4`, que activa la bandera Z cuando `x0 == -4`.

5.9 `print` y `error`

`print` es la rutina de salida común: recibe en `x1` la dirección del texto y en `x2` la longitud, y ejecuta `write(1, x1, x2)`. `error` informa un fallo de lectura con el mensaje `Opción inválida` y regresa al prompt.

6. Syscalls empleadas

Los números corresponden a la tabla de syscalls de Linux para ARM64 (distinta a la de x86-64). El número va en `x8`, los argumentos en `x0` – `x5`, se invoca con `svc #0` y el resultado vuelve en `x0` (negativo indica `-errno`).

Syscall	N.º (`x8`)	Argumentos	Dónde se usa
`read`	63	`x0`=0 (stdin), `x1`=buffer, `x2`=bytes	`read`, `verificar_largo` (descarte byte a byte)
`write`	64	`x0`=1 (stdout), `x1`=texto, `x2`=longitud	`print` (prompt, help, echo, clear, errores)
`exit`	93	`x0`=código	`exit` (código 0), hijo fallido (código 1)
`clone`	220	`x0`=17 (SIGCHLD), resto 0	`fork`
`execve`	221	`x0`=ruta, `x1`=argv, `x2`=NULL	`llamar_execve`
`wait4`	260	`x0`=pid, `x1`=NULL, `x2`=0, `x3`=NULL	`wait`

Se usan seis syscalls distintas. Todas se invocan con los números y parámetros propios de ARM64 Linux; `fork` se resuelve con `clone` porque esa arquitectura no define `fork` ni `wait` (solo `wait4`).

7. Uso del stack

7.1 Por qué es necesario

La instrucción `bl` guarda la dirección de retorno en `x30` (el link register). Si una función llama a otra con `bl` sin preservar `x30`, la dirección de retorno original se pierde y `ret` regresa al lugar equivocado. Durante el desarrollo este problema apareció en la práctica: una función con varios `bl` anidados no retornaba correctamente y el programa repetía salidas o se reiniciaba hasta que se incorporó el guardado del stack.

7.2 Patrón utilizado

Todas las funciones que llaman a otras abren y cierran un marco de 16 bytes con el mismo par de instrucciones:

```
funcion:
    stp  x29, x30, [sp, #-16]!   // pre-índice: sp -= 16, luego guarda x29 y
x30
    ...                          // cuerpo (puede hacer bl a otras funciones)
    ldp  x29, x30, [sp], #16     // post-índice: restaura x29 y x30, luego sp
+= 16
    ret
```

Se reserva siempre de a 16 bytes porque ARM64 exige que `sp` esté alineado a 16 bytes cuando se usa para acceder a memoria. El `!` (pre-índice) y el desplazamiento posterior (post-índice) garantizan que el guardado y la restauración sean simétricos, de modo que el stack queda balanceado al salir de cada función.

7.3 Funciones y su tratamiento del stack

Función	Llama a otras	Marco con STP/LDP
`verificar_largo`	`print`	Sí
`intrucion_init`	No (pero usa `x29` / `x30` por uniformidad)	Sí
`comparar`	`echo`, `clear_Command`, `help`, `exit`, `fork`	Sí
`help`, `echo`, `clear_Command`	`print`	Sí

Función	Llama a otras	Marco con STP/LDP
`fork`	`wait`, `execve`, `print`	Sí
`wait`	No	Sí
`execve`	`preparar_argumentos`, `construir_ruta`, `llamar_execve`	Sí
`print`, `read`, `llamar_execve`, `construir_ruta`, `preparar_argumentos`	No (funciones hoja)	No
`exit`, `error`	No retornan (terminan el proceso o saltan a `_start`)	No

Las funciones hoja no llaman a nadie, por lo que `x30` no se modifica y no necesitan guardarlo. Las funciones que nunca retornan tampoco requieren marco.

7.4 Registros callee-saved y decisiones de diseño

Según la convención AAPCS64, `x19` – `x28` son registros callee-saved. En este programa `x19` (longitud del mensaje) y `x22` (puntero al buffer de entrada) se escriben en `intrucion_init` sin restaurarse al salir, y `x16`, `x17` y `x18` se usan como estado global del ciclo. Es una decisión deliberada y segura en este contexto: el único llamador de esas funciones es `_start`, el programa no usa libc ni código externo que dependa de esos registros, y mantener el estado en registros evita variables en memoria. En un programa que se enlazara con otras bibliotecas habría que guardar esos registros con STP/LDP.

7.5 Verificación del balance del stack

En la sesión de GDB se comprobó que `sp` tiene exactamente el mismo valor en los tres puntos del ciclo (`print`, `intrucion_init` y `comparar`), lo que demuestra que cada vuelta del ciclo deja el stack igual que como lo encontró. Ver la sección 10.

8. Control de flujo y banderas de condición

Todo el control de flujo se logra con `cmp` / `cmn` (que actualizan las banderas N, Z, C y V) seguidos de saltos condicionales, o con `cbz` / `cbnz`, que comparan contra cero sin tocar las banderas.

Instrucción	Condición	Ejemplo en el código
`b.eq` / `b.ne`	Z = 1 / Z = 0	Comparar un byte contra espacio (32), tab (9) o `\n` (10); comparar el nombre del comando
`b.lt`	N ≠ V (menor con signo)	Detectar error de `read` / `clone` (resultado negativo) y la línea larga (`-1`)
`b.hs`	C = 1 (mayor o igual sin signo)	Fin de los datos: `cmp x15, x9` en el parser; tope de 63 caracteres
`b.lo`	C = 0 (menor sin signo)	Ciclos de comparación byte a byte: `cmp x15, #largo`
`cbz` / `cbnz`	Registro = 0 / ≠ 0	EOF (`cbz x0`), línea vacía (`cbz x18`), fin de cadena NUL en `execve`
`cmn x0, #4`	Z = 1 si `x0 == -4`	Reintento de `wait4` ante EINTR

Se usan comparaciones con signo (`b.lt`) para los códigos de error de las syscalls, que son negativos, y sin signo (`b.hs` , `b.lo`) para índices y longitudes, que nunca son negativos.

9. Manejo de errores y casos límite

Situación	Comportamiento
Línea vacía o solo espacios	No se ejecuta nada; se muestra un nuevo prompt
Comando desconocido (por ejemplo `noexiste`)	El hijo no logra hacer `execve` , imprime `No se pudo ejecutar el comando` y termina con código 1; el shell sigue vivo
Línea de más de 62 caracteres	Se descarta toda la línea, se muestra el aviso y se vuelve al prompt sin ejecutar nada
Ctrl+D (EOF)	Imprime un salto de línea y termina con `exit`
Falla de `read`	Muestra `Opción inválida` y regresa al prompt

Situación	Comportamiento
Falla de `clone`	Muestra `No se pudo crear el proceso hijo` y regresa al prompt
Espacios o tabs múltiples	El parser los ignora antes del comando y entre el comando y el mensaje
Ruta explícita (`/bin/echo hola`)	Se usa directamente sin anteponer `/bin/`

Limitaciones conocidas

- Los comandos externos se buscan únicamente en `/bin/` (o por ruta explícita con `/`); no se recorre la variable `PATH` .
- `execve` recibe `envp = NULL` , por lo que los programas externos se ejecutan sin variables de entorno.
- El tamaño máximo de línea es 62 caracteres y el máximo de argumentos está acotado por el arreglo `argumentos` (520 bytes).
- No se implementó el alcance opcional (pipes, redirecciones, historial, SIGINT, variables de entorno).

10. Evidencia de depuración con GDB

La depuración se hace con `gdb-multiarch` conectado a QEMU. En una terminal se ejecuta `make debug` (QEMU queda esperando en el puerto 1234) y en otra `make gdb` , que carga el script `s.gdb` . El script fija una arquitectura aarch64, se conecta con `target remote :1234` , coloca breakpoints en cinco funciones (`print` , `intrucion_init` , `comparar` , `fork` , `execve`) e inspecciona en cada parada los registros relevantes (`info registers`) y el stack (`x/8gx $sp`). Para guardar el log completo se ejecuta antes `script -a gdb_log.txt` .

10.1 Extracto de la sesión

Breakpoint en `print` (prompt): `x1` apunta al mensaje, `x2 = 0xf` (15 bytes) y `x30` contiene la dirección de retorno al ciclo principal.

```
Breakpoint 1, print () at practica.s:483
x1   0x4105f0   4261360
x2   0xf        15
x30  0x400468   4195432
sp   0x7c69cfffee40
```

Breakpoint en `intrucion_init`: `x0 = 0xa` (10 bytes leídos, por ejemplo `echo hola` más `\n`), `x16` y `x17` apuntan a los buffers de comando y mensaje. Antes del `stp`, el stack en `sp - 16` aún no contiene la dirección de retorno. Después de un `stepi`, la posición `0x7c69cfffee30` guarda `x29 = 0` y `x30 = 0x400488`, es decir, la dirección de retorno a `_start`:

```
(antes)  sp = 0x7c69cfffee40
stepi    // ejecuta: stp x29, x30, [sp, #-16]!
x/8gx $sp
0x7c69cfffee30: 0x0000000000000000  0x0000000000400488   <- x29, x30
guardados
0x7c69cfffee40: 0x0000000000000001  ...
```

Breakpoint en `comparar`: `x16` es la dirección del comando, `x18 = 4` (longitud de `echo`), `x19 = 4` (longitud del mensaje `hola`), `x30 = 0x400490` (retorno a `_start`).

Balance del stack: en las tres paradas anteriores, `sp` vale `0x7c69cfffee40`. Es decir, después de ejecutar `print`, `intrucion_init` y volver al ciclo, el puntero de pila regresa al mismo valor, lo que confirma que el guardado y la restauración con STP/LDP son simétricos.

> Nota: las direcciones `corresponden a la ejecución real en la máquina del autor (QEMU user-mode) y pueden variar en otro equipo. Al ejecutar ls, el breakpoint en fork se alcanzó y GDB se desprendió del proceso hijo (Detaching after fork from child process), por lo que no se detuvo en execve`.

Captura 1. Inicio de la sesión: los 5 breakpoints (`print`, `intrucion_init`, `comparar`, `fork`, `execve`), registros y stack iniciales, y la parada en `print` con `x1`, `x2` y `x30`.

```
daniel@DESKTOP-5SA89G3:~/arqui1$ make gdb
gdb-multiarch -q -x s.gdb programa
Reading symbols from programa...
The target architecture is set to "aarch64".

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at practica.s:386
386             ldr x1, =msg_inicial
Breakpoint 1 at 0x400538: file practica.s, line 483.
Breakpoint 2 at 0x4001ac: file leer_comando.s, line 24.
Breakpoint 3 at 0x4000c8: file interpretar.s, line 23.
Breakpoint 4 at 0x4003c4: file fork.s, line 17.
Breakpoint 5 at 0x4002d4: file execve.s, line 17.

=== Inicio: registros y stack ===
x0              0x0                 0
x1              0x0                 0
x2              0x0                 0
x8              0x0                 0
x30             0x0                 0
sp              0x7c69cfffee40      0x7c69cfffee40
0x7c69cfffee40: 0x0000000000000001      0x00007c69cffff0c9
0x7c69cfffee50: 0x0000000000000000      0x00007c69cffff0d4
0x7c69cfffee60: 0x00007c69cffff0fc      0x00007c69cffff104
0x7c69cfffee70: 0x00007c69cffff11c      0x00007c69cffff15d

Breakpoint 1, print () at practica.s:483
483             mov x0, #1              // stdout

=== Parada en print (prompt): x1 = mensaje, x2 = longitud ===
x0              0x0                 0
x1              0x4105f0            4261360
x2              0xf                 15
x8              0x0                 0
x30             0x400468            4195432
sp              0x7c69cfffee40      0x7c69cfffee40
0x7c69cfffee40: 0x0000000000000001      0x00007c69cffff0c9
0x7c69cfffee50: 0x0000000000000000      0x00007c69cffff0d4
0x7c69cfffee60: 0x00007c69cffff0fc      0x00007c69cffff104
0x7c69cfffee70: 0x00007c69cffff11c      0x00007c69cffff15d

Breakpoint 2, intrucion_init () at leer_comando.s:24
24          stp x29, x30, [sp, #-16]!
```

Captura 2. Paradas en `intrucion_init` (antes y después de `stepi`, con `x29` y `x30` guardados en el stack) y en `comparar` (`x16`, `x18`, `x19`). Después se ve el ciclo de `print` y el nuevo ingreso a `intrucion_init`.

```
=== Parada en intrucion_init: x0 = bytes leidos ===
x0             0xa                10
x9             0x0                0
x16            0x410748           4261704
x17            0x410788           4261768
x30            0x400488           4195464
sp             0x7c69cfffee40     0x7c69cfffee40
0x7c69cfffee40: 0x0000000000000001      0x00007c69cffff0c9
0x7c69cfffee50: 0x0000000000000000      0x00007c69cffff0d4
0x7c69cfffee60: 0x00007c69cffff0fc      0x00007c69cffff104
0x7c69cfffee70: 0x00007c69cffff11c      0x00007c69cffff15d
26          mov x9, x0
0x7c69cfffee30: 0x0000000000000000      0x0000000000400488
0x7c69cfffee40: 0x0000000000000001      0x00007c69cffff0c9
0x7c69cfffee50: 0x0000000000000000      0x00007c69cffff0d4
0x7c69cfffee60: 0x00007c69cffff0fc      0x00007c69cffff104
(arriba: el stp ya guardo x29 y x30 en el stack)

Breakpoint 3, comparar () at interpretar.s:23
23          stp x29, x30, [sp, #-16]!

=== Parada en comparar: x16 = comando, x18 = longitud ===
x16            0x410748           4261704
x17            0x410788           4261768
x18            0x4                4
x19            0x4                4
x30            0x400490           4195472
sp             0x7c69cfffee40     0x7c69cfffee40
0x7c69cfffee40: 0x0000000000000001      0x00007c69cffff0c9
0x7c69cfffee50: 0x0000000000000000      0x00007c69cffff0d4
0x7c69cfffee60: 0x00007c69cffff0fc      0x00007c69cffff104
0x7c69cfffee70: 0x00007c69cffff11c      0x00007c69cffff15d
(gdb) continue
Continuing.

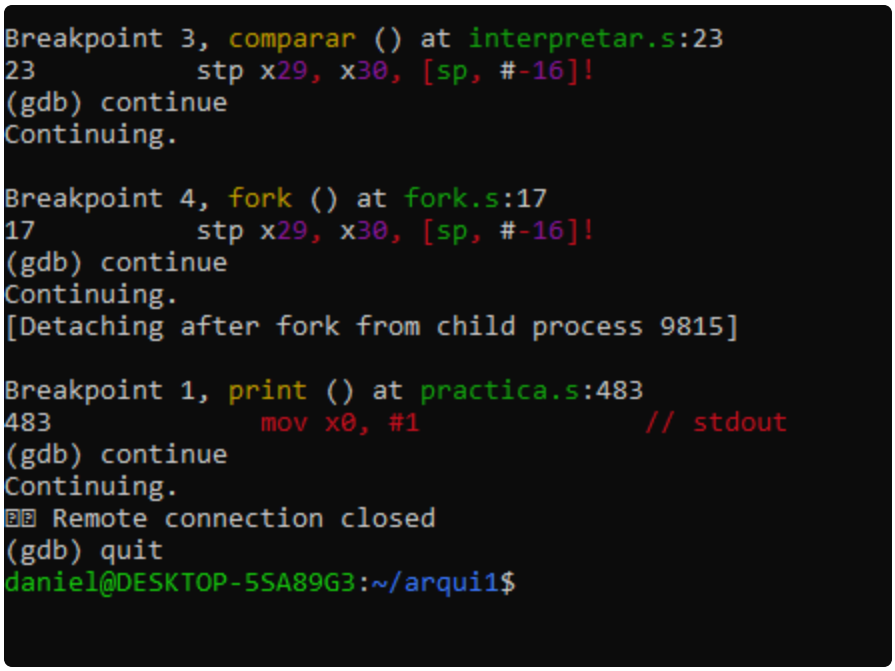
Breakpoint 1, print () at practica.s:483
483             mov x0, #1              // stdout
(gdb) continue
Continuing.

Breakpoint 1, print () at practica.s:483
483             mov x0, #1              // stdout
(gdb) continue
Continuing.

Breakpoint 1, print () at practica.s:483
483             mov x0, #1              // stdout
(gdb) continue
Continuing.

Breakpoint 2, intrucion_init () at leer_comando.s:24
24          stp x29, x30, [sp, #-16]!
(gdb) continue
Continuing.
```

Captura 3. Parada en `comparar` y en `fork` al ejecutar un comando externo; GDB se desprende del proceso hijo y la sesión termina con `quit`.



10.2 Qué demuestra la sesión

- Inspección del stack con `x/8gx $sp` antes y después de `stp`.
- Visualización de registros con `info registers` en cada función.
- Breakpoints en más de tres funciones del programa.
- Seguimiento de la ejecución con `continue` y `stepi`.

11. Pruebas realizadas

El archivo `pruebas.sh` envía líneas al shell con una pausa entre cada una (como si se tecleara) y busca el texto esperado en la salida. Se ejecuta con `make test`. Las 20 pruebas pasan:

Categoría	Pruebas
Prompt y salida	Se muestra `OMEGA_TERMINAL>`; `exit` termina
Builtins	`help`; `echo hola mundo`; `echo` con espacios extra; `echo` con tab; espacios antes del comando; `clear` (secuencia ANSI `ESC[2J`)

Categoría	Pruebas
Externos	`pwd`; `ls -l /`; `date`; `/bin/echo directo`
Errores	Comando desconocido imprime el mensaje; el shell sigue vivo después; línea vacía no ejecuta y el shell sigue vivo
Línea larga	Imprime el aviso, no ejecuta el resto y sigue vivo
EOF	Ctrl+D termina limpio (no se cicla)

Además de la rúbrica del proyecto (mínimo 8 casos: prompt, lectura, parsing, help, clear, exit, echo y un comando externo), la suite cubre los casos límite del parser y de los errores.

12. Problemas encontrados y soluciones

Problema	Causa	Solución
El programa imprimía muchas veces el mismo resultado y se reiniciaba	Funciones con `bl` anidados sin guardar `x30`	Uso de STP/LDP en todas las funciones que llaman a otras
Aparecían números de más al imprimir	El `\n` de la lectura contaba como un byte del mensaje y la longitud era incorrecta	El parser termina en `\n` y calcula la longitud real del mensaje
`print` alteraba datos de otras funciones	La syscall modifica `x0` y se usaba `x1` como almacén	Mensaje en `x17` (`mensaje_buffer`) y longitud en `x19`, ambos fijos
Una línea larga se ejecutaba por partes	El resto quedaba en el terminal y se leía como comando nuevo	`verificar_largo` descarta hasta el siguiente `\n`
Ctrl+D hacía que el shell se ciclara	`read` devolvía 0 y el ciclo seguía	Se comprueba `cbz x0` tras `read` y se llama a `exit`
`fork` no existe en ARM64	La arquitectura solo define `clone`	`clone(SIGCHLD)` con el número 220

Problema	Causa	Solución
Errores de ensamblado y enlace	Mnemónicos mal escritos (`cpm` en lugar de `cmp`), faltaba la nueva línea final y había referencias a etiquetas inexistentes	Corrección de los símbolos y de las referencias entre módulos

13. Conclusiones

MiniShell cumple el alcance obligatorio del proyecto: prompt con `write` y `read`, parsing modular, cuatro comandos internos, ejecución de programas externos con `clone` y `execve`, manejo de errores, Makefile funcional y código comentado, con más de cinco funciones que usan STP/LDP para preservar el link register.

El proyecto permitió aplicar de forma práctica el ISA ARM-64: la convención de llamada (parámetros en `x0` – `x7`, `x8` para el número de syscall), el manejo del stack con alineación de 16 bytes, las banderas de condición para decisiones y ciclos, y la interacción directa con el kernel mediante syscalls. Como trabajo futuro quedan las funciones opcionales (pipes con `pipe`, redirección con `dup3`, historial, manejo de SIGINT y búsqueda en `PATH`).

14. Referencias

- Linux man pages: https://man7.org/linux/man-pages/
- Tabla de syscalls de Linux para ARM64 (`asm-generic/unistd.h`).
- Procedure Call Standard for the Arm 64-bit Architecture (AAPCS64).
- Documentación de GNU Assembler (GAS) y de GDB.